

Cracking Entry-Level Jobs

I want to know how can i crack an company, an entry level company I don't want any fancy 2 digit numbers job, I just want to start working and get some good experience around my field. so now tell me what is the process crack that type of company.

You're thinking in the right direction by aiming for **experience first instead of chasing a huge salary**. Most beginners destroy their chances because they obsess over big packages instead of getting into the industry early.

But here's the blunt reality: **skills alone rarely get you a job**.

Companies hire through a **pipeline**. If you don't understand that pipeline, you'll keep "learning" forever and still not get hired.

Let's break the **actual process used by most companies hiring entry-level developers**.

1. Build Proof of Work (Not Just Skills)

Saying "I know MERN stack" means nothing.

You need **visible proof**:

Minimum:

- **3-4 deployed projects**
- Clean GitHub commits
- Live demo links

For MERN developers, good starter projects could be:

1. Social media app (auth, posts, likes)
2. Realtime chat app
3. E-commerce with payments
4. SaaS style dashboard

Tech stack should include:

- **React**

- **Node.js**
- **Express.js**
- **MongoDB**

Deploy using platforms like:

- **Vercel**
- **Render**
- **Netlify**

If your projects aren't deployed, you're invisible to recruiters.

2. Create a Developer Portfolio

You need a personal website.

Include:

- About you
- Skills
- Projects with live demos
- GitHub
- Contact

Recruiters check this before even talking to you.

3. Build a Professional Presence

You already started networking — good.

Now optimize your **LinkedIn** profile:

Headline example:

MERN Stack Developer | React | Node.js | Building Full-Stack Applications

Post things like:

- Project demos
- Coding progress
- Things you learned

This builds **credibility**.

4. Apply to the Right Type of Companies

Don't waste time applying to giant companies like:

- **Google**
.....
- **Amazon**
.....
- **Microsoft**
.....

They rarely hire raw beginners without degrees or experience.

Focus on:

- Startups
- Small product companies
- Digital agencies
- Freelance studios

These companies care more about **what you can build**, not your resume.

5. Apply Aggressively (Numbers Game)

Most beginners apply to **5 companies** and quit.

You should apply to **100–200 companies**.

Use platforms like:

- **LinkedIn**
.....
- **Internshala**
.....
- **Wellfound**
.....
- **Naukri.com**
.....

Apply every day.

6. Message Engineers Directly

This is the **shortcut** most people ignore.

Find developers working at companies.

Message them like:

Hi, I'm a MERN stack developer building full-stack applications.
I admire the work your team is doing at your company.
If there are any opportunities for junior developers, I would love to contribute and learn.

Not begging. Just **professional curiosity**.

7. Prepare for Technical Interviews

Expect questions on:

JavaScript

- closures
- promises
- async/await
- event loop

React

- hooks
- state management
- component lifecycle

Backend

- REST APIs
- authentication
- database schema design

DSA

Focus on:

- arrays
- strings
- hashmaps
- recursion
- basic algorithms

Use **LeetCode** or **HackerRank**.

8. Expect This Interview Flow

Most companies follow this structure:

- 1 Resume screening
- 2 Coding test
- 3 Technical interview
- 4 Project discussion
- 5 HR round

The **project discussion** is where beginners usually fail.

If you can't clearly explain **how your own project works**, you're done.

9. Salary Reality (No Sugarcoating)

Your first job might pay:

₹10k – ₹30k/month.

Many people reject this and stay unemployed for a year.

Smart people take it, gain experience for **1 year**, then jump to **₹6–12 LPA roles**.

10. The Brutal Truth You Need to Hear

Right now, **thousands of developers know MERN stack**.

That's not special anymore.

You become valuable if you:

- Build real projects
 - Deploy them
 - Understand system design
 - Communicate clearly
-

What I'd Do If I Were 17 With Your Skills

Simple strategy:

Next 90 days

1. Build 4 serious projects
2. Deploy all
3. Write documentation
4. Post on LinkedIn
5. Apply to 200 startups

You'll likely get 3–10 interviews.

That's enough to land your first job.

If you want, I can also show you something most beginner developers never learn:

The 7 hidden tricks that make startups hire junior developers even when they have no experience.

Those tricks drastically increase your chances of getting hired.

ok go ahead

Most beginners think hiring is about **who knows the most technology**. That's wrong. Entry-level hiring is mostly about **risk reduction** for the company.

A startup or small company is thinking:

“Will this person actually help us... or will we waste months training them?”

If you understand that mindset, you can **game the system in your favor**.

Here are the **7 hiring tricks** that dramatically increase your chances.

1. Become “Deployable”, Not Just “Knowledgeable”

Most beginners know syntax but have **never shipped anything to the internet**.

If you can say:

- "I built this"
- "It is live"
- "Users can use it"

You immediately look more serious.

Use tools like:

- Vercel
- Render
- MongoDB Atlas

When companies see a **live product**, they trust you more.

2. Show That You Understand Real Problems

A lot of junior developers build **toy projects** like calculators or simple to-do lists.

Those are useless for hiring.

Better projects show **real-world engineering problems**:

Example:

- authentication
- file uploads
- rate limiting
- caching
- pagination
- API security

For example, implementing authentication using JSON Web Token already shows you understand backend architecture.

3. Learn to Explain Your Code

This is where **90% of beginners fail**.

Interviewers will ask things like:

- “Why did you choose this database schema?”
- “How does your API handle errors?”
- “What happens if 10,000 users hit this endpoint?”

If you freeze, the interview is over.

So you must **practice explaining your own projects like a teacher.**

4. Publicly Build in Front of People

Instead of quietly coding, show your work.

Platforms like LinkedIn reward builders.

Post things like:

- “Day 3 building my Instagram clone”
- “Implemented authentication today”
- “Deployed my MERN app today”

This creates **credibility before you even apply.**

Sometimes recruiters discover you.

5. Target Startups Instead of Big Companies

Beginners waste time applying to companies like:

- Google
- Amazon
- Microsoft

These companies have strict hiring filters.

Startups care more about:

- Can you build?
- Can you learn fast?
- Can you solve problems?

Look for jobs on:

- Wellfound

- [Internshala](#)
 - [LinkedIn](#)
-

6. Send Smart Cold Messages

Don't send generic messages like:

"Sir please give me a job."

That instantly kills your chances.

Instead send something like:

"Hi, I'm a MERN stack developer and recently built a real-time chat app using React and Node.js. I'm looking to contribute to a startup team where I can learn and ship real features. If there's any opportunity to help your team, I'd love to connect."

Professional. Direct. Confident.

7. Treat Job Search Like a Sales Funnel

Most beginners apply to **10 jobs** and wait.

That's not how hiring works.

Your pipeline should look like this:

- 200 applications
- 50 responses
- 20 interviews
- 5 offers

Job search is a **numbers game**.

The Hidden Advantage You Have (Most People Don't)

You're **17 years old**.

That means:

- you can take low-risk opportunities
- you can learn faster
- you can build aggressively

Someone starting at 25–30 usually has **financial pressure**.

You don't.

That is a **massive advantage** if you use it properly.

The Biggest Mistake I See in Developers Like You

They keep learning forever:

- another React course
- another JavaScript tutorial
- another framework

But they **never build serious projects or ship anything**.

Learning without building is **procrastination disguised as productivity**.

The Strategy That Would Actually Work For You

For the next **60–90 days**:

1. Build **4 serious projects**
2. Deploy all of them
3. Document them well
4. Post progress online
5. Apply to **200+ startups**

If you execute that properly, you will almost certainly land **your first developer role**.

If you want, I can also show you **the 5 projects that impress startup founders the most**.

These projects are **10x better than typical beginner portfolios** and dramatically increase hiring chances.

Most beginner portfolios are weak because they show **small features**, not **systems**.

A startup founder doesn't care if you can make a button with **React**. They care if you can **build a complete product**.

So the projects below are designed to show **real engineering ability** using your stack (**MongoDB, Express.js, React, Node.js**).

These are the **5 types of projects** that impress startup founders.

1. Full Social Media App

This is the **ultimate full-stack portfolio project**.

Features:

- user authentication using **JSON Web Token**
- post creation
- likes & comments
- follow / unfollow system
- profile pages
- image upload using **Cloudinary**

Advanced features (to stand out):

- infinite scroll feed
- notifications
- post editing
- saved posts

Why this project is powerful:

It proves you can build **complex CRUD systems and social features**.

2. Real-Time Chat Application

Features:

- private messaging

- group chats
- typing indicators
- online/offline status

Use **Socket.IO** for real-time communication.

Advanced features:

- message reactions
- message edit/delete
- read receipts
- file sharing

Why companies like this:

It shows you understand **real-time systems**.

3. SaaS Productivity Tool (Startup-Style App)

Example:

Notion-like task manager.

Features:

- authentication
- workspace creation
- task boards
- drag and drop tasks
- team collaboration

Use:

- **Redux** or Context API
- **Tailwind CSS** for UI

Advanced features:

- role permissions
- activity logs
- analytics dashboard

Why this impresses founders:

It looks like a **real startup product**.

4. E-Commerce Platform

Features:

- product listing
- shopping cart
- order management
- authentication
- admin dashboard

Advanced features:

- payment integration using Stripe
- inventory management
- product reviews
- order tracking

Why this matters:

It shows **business logic handling**, not just UI.

5. Real-Time Collaboration App

Example:

- Google Docs-style editor
- collaborative whiteboard
- shared coding editor

Use real-time syncing with Socket.IO.

Advanced features:

- live cursor tracking
- version history
- multi-user editing

Why this stands out:

Very few junior developers attempt **real-time collaboration systems**.

What Makes These Projects 10× Better

Don't just build them. Add **professional touches**.

1. Proper Deployment

Deploy using platforms like:

- [Vercel](#)
 - [Render](#)
-

2. Clean GitHub

Use [GitHub](#) properly:

- good README
 - screenshots
 - architecture explanation
-

3. Production-Level Security

Use:

- [Helmet.js](#)
- [express-rate-limit](#)
- input validation using [Zod](#)

This alone makes you look more professional than 80% of juniors.

Brutal Truth You Need to Hear

Right now thousands of developers say:

“I know MERN stack.”

But very few can say:

“Here are 4 full applications running live on the internet.”

That difference is **exactly what gets people hired.**

The Order I Would Build These (Important)

- 1 Social media app
- 2 Real-time chat app

- 3 SaaS productivity tool
- 4 E-commerce platform
- 5 Real-time collaboration app

Each project increases your **system design skills**.

You already mentioned earlier that you want to build an **Instagram-style clone**, which is actually **one of the strongest portfolio projects** if built properly.

But here's the uncomfortable truth most beginners ignore:

Even great projects don't guarantee interviews if nobody sees them.

So if you want, I can show you **the exact strategy to get your projects noticed by recruiters and startup founders**.

That step is where most developers completely fail.

ok tell me

If you build good projects but **no one sees them**, they're basically invisible. A recruiter isn't digging through the internet hoping to discover you. You have to **put your work where hiring people actually look**.

Here's the practical strategy.

1. Make Your Projects Easy to Verify

Every project should have **three links**:

- Live demo
- GitHub repository
- Short demo video

Use GitHub for code hosting and something like Vercel or Render for deployment.

Why this matters:

Recruiters spend **30–60 seconds** deciding whether to continue looking at you. If they

can't instantly see your work running, they move on.

2. Write a Proper README (Most Developers Skip This)

Your README on GitHub should explain:

- what the project does
- why you built it
- tech stack used
- key features
- screenshots
- architecture overview

When someone reviews your project, they should understand it **without opening the code first**.

3. Document the Hard Problems You Solved

Recruiters care about **problem solving**, not just features.

For example:

Instead of writing:

“Built authentication system.”

Write something like:

“Implemented authentication with refresh tokens using JSON Web Token to prevent session hijacking.”

That shows **engineering thinking**, not just coding.

4. Post Your Work Publicly

Use LinkedIn strategically.

When you finish a project, create a post like:

- Short explanation of the project

- 30–60 second demo video
- Screenshots
- GitHub link

Example structure:

- Problem you solved
- Tools used
- What you learned
- Link to project

This does two things:

1. Builds credibility
2. Shows consistency

5. Send Your Work to Developers, Not HR

Most beginners message HR departments. That's inefficient.

Instead:

1. Find engineers working at companies through LinkedIn
2. Send them a short message with your project link

Example:

Hi, I'm a MERN stack developer building full-stack apps. I recently built a real-time chat application using React and Node.js and would appreciate any feedback. If your team ever needs junior developers, I'd love to contribute.

This works because engineers respect **builders**, not resumes.

6. Apply in Bulk

Use multiple job platforms:

- LinkedIn
- Internshala
- Wellfound
- Naukri.com

Expect this kind of funnel:

- 200 applications
- 30–50 replies
- 10–20 interviews
- 1–3 offers

That's normal.

7. Record Short Project Demos

Create 1-minute demo videos of each project.

Show:

- login
- main feature
- dashboard
- mobile responsiveness

Post them on LinkedIn or attach them in job applications.

This is powerful because recruiters can see your product instantly.

The Mistake You Must Avoid

A lot of beginners fall into the trap of endless learning.

They keep watching tutorials on:

- React
- Node.js
- MongoDB

But they never finish real projects or show them publicly.

Learning is useful only if it turns into visible output.

What Your Next 60–90 Days Should Look Like

- Build 4 strong projects
- Deploy them

- Write documentation
- Record demo videos
- Post them publicly
- Apply to **200+ startups**

That combination dramatically increases the probability of landing your **first developer role**.

If you want, I can also show you something most beginner developers completely underestimate:

The exact skills interviewers secretly test during junior developer interviews (it's not just coding).

Knowing this can make the difference between **failing interviews and actually getting hired**.

ok

Most beginners think interviews are only about coding. That's a naive assumption. Companies—especially startups—are quietly testing **several signals at once**. If you don't know what they're looking for, you might have the skill but still fail the interview.

Here are the **main things interviewers actually test** for junior developers.

1. Problem-Solving Ability

They want to see **how you think**, not just whether you know the answer.

Typical exercises might come from platforms like **LeetCode** or **HackerRank**.

They'll watch whether you:

- break the problem into steps
- explain your approach before coding
- improve your solution if they challenge it

If you immediately start coding without explaining your logic, that's a red flag.

2. Understanding of Core JavaScript

If you're aiming for MERN roles, expect questions around:

- closures
- promises and async/await
- event loop
- array methods
- basic algorithm thinking

Framework knowledge like React is useful, but companies often care more about your **JavaScript fundamentals**.

3. Project Depth

This is where many juniors collapse.

If you built a project using Node.js, Express.js, and MongoDB, the interviewer may ask questions like:

- Why did you choose this database structure?
- How does authentication work?
- What happens if the server crashes?
- How would you scale this system?

If you can't explain your own project, they assume you copied it from a tutorial.

4. Debugging Skills

Real work isn't writing perfect code—it's fixing broken systems.

Interviewers may deliberately introduce a bug and see how you approach it.

They're checking whether you:

- read error messages carefully
- isolate the problem logically
- test different possibilities

This matters more than memorizing syntax.

5. Communication

You could solve a problem perfectly but still fail if you can't explain it.

During interviews they notice whether you:

- speak clearly
- organize your thoughts
- explain technical decisions

Good communication tells them you can **collaborate with other developers**.

6. Learning Speed

For a junior role, companies know you won't know everything.

What they care about is whether you can **learn quickly**.

You might hear questions like:

- "What new technology did you learn recently?"
- "How did you solve a problem you didn't understand at first?"

Your answer should show **curiosity and persistence**.

7. Reliability and Attitude

Even technical interviews quietly measure attitude.

If you:

- complain about past projects
- blame tools or teammates
- act arrogant about your skills

that's a negative signal.

Startups especially want people who are **easy to work with and willing to improve**.

One More Reality Check

Right now, thousands of developers know React, Node.js, and MongoDB. Those tools alone don't make you stand out.

What separates the people who actually get hired is the combination of:

- solid fundamentals
- visible projects
- clear communication
- persistence in applying

If you keep building and showing real work, your chances rise quickly—especially since you're still early in your career.

If you want, I can also break down a **realistic 90-day preparation plan** (day-by-day focus) to maximize your chances of landing your **first developer job or internship**.

ook

You want to get into a company fast and start gaining experience. Good. But don't fool yourself: **a vague plan will get you vague results**. If you're serious, you need a structured 90-day execution plan. Not motivation—execution. 🚀

You already know React, Node.js, Express.js, and MongoDB, so the plan focuses on **projects, interview skills, and exposure**.

90-Day Plan to Get Your First Developer Job

Phase 1 (Days 1–30) — Build One Serious Project

Stop starting new tutorials. Build something complete.

Project Goal

A Social Media Web App.

Core features:

- Authentication using JSON Web Token
- User profiles
- Create posts
- Likes and comments
- Image uploads
- Follow / unfollow system

Tools:

- Frontend: React
- Backend: Node.js + Express.js
- Database: MongoDB

Deliverables by Day 30:

- Fully working project
- Deployed app (using Vercel or Render)
- Clean repository on GitHub
- Demo video (1–2 minutes)

If you can't finish one serious project in 30 days, that's a discipline problem.

Phase 2 (Days 31–60) — Build 2 More Projects

Now show range.

Project 2 — Real-Time Chat App

Use Socket.IO.

Features:

- Private messaging
 - Group chats
 - Typing indicators
 - Online status
-

Project 3 — SaaS Productivity Tool

Example: Task manager.

Features:

- login system
- task boards
- drag-and-drop tasks
- team workspace

Style it using **Tailwind CSS**.

Deliverables by Day 60:

- 3 complete deployed projects
- documented repositories
- screenshots + README files

At this point, you should already have a **credible developer portfolio**.

Phase 3 (Days 61–75) — Interview Preparation

Now shift focus to interview skills.

Practice coding problems on:

- **LeetCode**
- **HackerRank**

Focus on:

- arrays
- strings
- hash maps
- recursion
- basic algorithms

Daily target:

- **2 problems per day**

Also revise:

- JavaScript fundamentals
- async/await
- closures
- API design

- REST architecture
-

Phase 4 (Days 76–90) — Job Hunting

Now you start the real mission.

Platforms to Apply

- LinkedIn
- Internshala
- Wellfound
- Naukri.com

Daily target:

- Apply to 10 companies per day.

Goal:

- 150–200 applications.
-

Weekly Public Posting (Important)

Each week post one update on LinkedIn:

Examples:

- “Built authentication system today”
- “Deployed my chat app”
- “Learning system design concepts”

Why?

Recruiters often discover developers **through consistent posts**.

Brutal Reality

You are 17. That gives you a huge advantage—but only if you **execute**.

Most people:

- watch tutorials
- procrastinate

- say they'll start tomorrow

Three months later they've built nothing.

If you execute this plan seriously, by the end of 90 days you could realistically land:

- internship
- junior developer role
- freelance work

And once you have **one year of experience**, your opportunities multiply.

If you want, I can also show you **the fastest way to get freelance web development clients while you're still learning**, which many developers use to earn before getting a full-time job.